CSS Tutorial: Learn by Building

A hands-on guide to styling web pages with colors, typography, spacing, layouts, and motion. Every new idea is explained in one sentence and then shown in working code.

How to use this guide

1. Create a folder called `css-practice` and add an `index.html` file and a `styles.css` file.
2. Link the CSS file from the HTML file:

```
<link rel="stylesheet" href="styles.css">
```

3. Type each example yourself, save both files, then refresh the browser.
4. Change a value, break the rule on purpose, and see what happens. Use the browser's developer tools to inspect the result.

You need a browser (Chrome, Firefox, Edge, or Safari) and a text editor such as VS Code. This guide assumes that you have already learned basic HTML.

0. Your first stylesheet

CSS describes the presentation of HTML by matching elements and applying declarations. A CSS rule has a selector, followed by declarations inside braces; each declaration has a property and a value.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My Styled Page</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <h1>Hello, CSS!</h1>
    <p>This page is styled by an external stylesheet.</p>
  </body>
</html>
```

```
h1 {
  color: navy;
  font-size: 48px;
}

p {
  color: #333333;
```

```
  line-height: 1.6;
}
```

Try it: change `navy` to `tomato`, `48px` to `64px`, and refresh. Add the comment `/* This is a CSS comment */` anywhere in the stylesheet.

0.1 How to add CSS

External CSS should be your default because it keeps content and presentation separate. Internal CSS is useful for a single page, and inline CSS should normally be avoided because it is difficult to maintain.

```
<!-- External stylesheet -->
<link rel="stylesheet" href="styles.css">

<!-- Internal stylesheet -->
<style>
  body {
    background-color: white;
  }
</style>

<!-- Inline style -->
<p style="color: red;">Use this only for quick tests.</p>
```

---

1. Selectors and the cascade

1.1 Type, class, and ID selectors

Type selectors match elements, class selectors match reusable groups, and ID selectors match one unique element. Classes are usually the best choice because they are reusable and predictable.

```
<h1 id="page-title">Travel Guide</h1>
<p class="intro">Plan your next adventure.</p>
<p class="intro emphasized">Both classes apply to this paragraph.</p>
```

```
h1 {
  color: darkblue;
}

.intro {
  font-size: 18px;
}

.emphasized {
  font-weight: bold;
```

```
}

#page-title {
  text-align: center;
}
```

1.2 Descendant and child selectors

A descendant selector matches an element anywhere inside another element, while a child selector matches only its direct children.

```
<article>
  <p>Direct paragraph.</p>
  <div>
    <p>Nested paragraph.</p>
  </div>
</article>
```

```
article p {
  color: #333333;
}

article > p {
  font-weight: bold;
}
```

1.3 Pseudo-classes

Pseudo-classes describe an element's state or position. Common examples include hover, focus, checked, first-child, and last-child.

```
<a href="about.html">About us</a>
<input type="email" placeholder="Email address">
```

```
a:hover {
  color: white;
  background: navy;
}

input:focus {
  border: 2px solid dodgerblue;
  outline: none;
}
```

```
li:first-child {
  font-weight: bold;
}
```

1.4 Pseudo-elements

Pseudo-elements style a specific part of an element, such as its first line or first letter. `::before` and `::after` can insert decorative generated content.

```
<p class="lead">CSS is a powerful styling language.</p>
<a class="external-link" href="https://example.com">Visit the site</a>
```

```
.lead::first-letter {
  float: left;
  font-size: 48px;
}

.external-link::after {
  content: " ↗";
}
```

1.5 Attribute selectors

Attribute selectors target elements based on their attributes or attribute values.

```
<input type="email">
<input type="text">
<a href="https://example.com">Example</a>
```

```
input[type="email"] {
  border-color: purple;
}

a[href^="https://"] {
  color: green;
}
```

1.6 Specificity and the cascade

When several rules target the same property, the browser uses specificity to decide the winner; inline styles are strongest, IDs beat classes, classes beat element types, and later rules win when specificity is equal.

```css
p {
  color: gray;
}

.intro {
  color: blue;
}

#welcome {
  color: green;
}
```

```css
/* Use only as a last resort, because it is difficult to maintain. */
.intro {
  color: red !important;
}
```

1.7 Inheritance

Some properties are inherited from a parent, such as `color` and `font-family`, while properties such as borders and margins are not. `inherit` explicitly copies a value from the parent.

```css
body {
  color: #222222;
  font-family: Arial, sans-serif;
}

.button {
  color: inherit;
  border: 1px solid currentColor;
}
```

Practice 1: Style a profile page

Create an HTML profile page and style it with at least one element selector, one class selector, one ID selector, one descendant selector, a hover state, and an accessible focus state. Then check which CSS rule wins when two selectors conflict.

---

2. Values, color, and typography

2.1 CSS units

Use `px` for precise small details, `rem` for text and consistent spacing, `%` for values relative to a parent, and `vw`/`vh` for values relative to the viewport.

```
h1 {
  font-size: 3rem;
}

.container {
  width: 90%;
  max-width: 1200px;
}

.full-screen {
  min-height: 100vh;
}
```

2.2 Colors

CSS colors can be written as named colors, hex, RGB, HSL, or with an alpha channel.

```
.named {
  color: tomato;
}

.hex {
  background-color: #2563eb;
}

.rgb {
  color: rgb(17, 24, 39);
}

.hsl {
  border-color: hsl(210, 70%, 40%);
}

.transparent {
  background-color: rgb(37, 99, 235, 0.2);
}
```

Use colors with enough contrast for readable text. Text should normally meet WCAG AA contrast ratios of at least 4.5:1 for normal text and 3:1 for large text.

2.3 Fonts and text

`font-family` chooses a font stack, `font-size` sets text size, `font-weight` controls thickness, and `line-height` controls vertical spacing.

```
<article>
  <h1>Design Principles</h1>
```

```
  <p>Good typography makes content easier to understand.</p>
</article>
```

```
body {
  font-family: Arial, Helvetica, sans-serif;
}

h1 {
  font-size: 2.5rem;
  font-weight: 700;
}

p {
  font-size: 1rem;
  line-height: 1.6;
}
```

2.4 Text alignment and decoration

Text properties can align, decorate, transform, and control the spacing of text.

```
h1 {
  text-align: center;
  text-transform: uppercase;
  letter-spacing: 0.08em;
}

.old-price {
  text-decoration: line-through;
}

.underlined {
  text-decoration: underline;
  text-underline-offset: 4px;
}
```

2.5 Web fonts

A web font can be imported with `@import`, but the `link` method in HTML is often faster and simpler.

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?
family=Roboto:wght@400;700&display=swap" rel="stylesheet">
```

```
body {
  font-family: "Roboto", sans-serif;
}
```

Practice 2: Create a readable article

Style an article with a custom font stack, a large heading, a comfortable line length, line-height, heading alignment, body text color, and one highlighted word using a different color and weight.

---

3. The box model

3.1 Width, height, and the box

Every element is a box made of content, padding, border, and margin. In the default `content-box` model, width applies only to the content, so padding and border make the total width larger.

```
div {
  width: 300px;
  padding: 20px;
  border: 2px solid black;
  margin: 30px;
}
```

3.2 Border-box

Using `box-sizing: border-box` makes the declared width include padding and border, which usually creates more predictable layouts.

```
* {
  box-sizing: border-box;
}

.card {
  width: 300px;
  padding: 20px;
  border: 2px solid #dddddd;
}
```

3.3 Padding and margin

Padding spaces content inside a border, while margin spaces the element away from neighboring elements. Shorthand values follow top, right, bottom, and left order.

```
.card {
  margin: 24px 16px;
  padding: 20px 16px 20px 16px;
}

.section {
  margin-top: 64px;
}
```

3.4 Display types

`block` elements start on a new line and can have width and height, `inline` elements share a line and ignore vertical margin changes, and `inline-block` combines both behaviors.

```
article {
  display: block;
}

a {
  display: inline-block;
  padding: 10px 16px;
}
```

3.5 Borders and border radius

Borders can have a style, width, and color, while border radius rounds corners.

```
.card {
  border: 1px solid #d1d5db;
  border-radius: 12px;
}

.avatar {
  border-radius: 50%;
}
```

3.6 Overflow

Overflow controls what happens when content does not fit its box.

```
.long-box {
  height: 120px;
  overflow: auto;
}
```

```
.image-frame {
  overflow: hidden;
}
```

3.7 Shadows and outlines

Box shadows create depth, and outlines draw outside the border without changing layout.

```
.card {
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.12);
}

button:focus-visible {
  outline: 3px solid orange;
  outline-offset: 3px;
}
```

Practice 3: Build a card component

Build one HTML card containing an image, heading, paragraph, and button. Use border-box sizing, padding, margin, a border, rounded corners, a shadow, and a visible hover or focus effect.

---

4. Backgrounds, images, and gradients

4.1 Background colors and images

`background-color` adds a solid background, while `background-image` can place an image over it.

```
.hero {
  color: white;
  background-color: #1e293b;
  background-image: url("hero.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
}
```

4.2 Gradients

A gradient is generated by CSS and can create smooth color transitions without an image.

```
.banner {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
}

.soft-banner {
```

```
  background: radial-gradient(circle, white 0%, #dbeafe 100%);
}
```

4.3 Images

Images can be made responsive by limiting their maximum width, and object-fit can control how replaced content fills a frame.

```
img {
  max-width: 100%;
  height: auto;
}

.profile-image {
  width: 160px;
  height: 160px;
  object-fit: cover;
}
```

---

5. Layout with Flexbox

5.1 Flex container and items

Flexbox arranges direct children along a main axis. Set `display: flex` on a parent to turn its children into flex items.

```
<nav class="nav">
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Contact</a>
</nav>
```

```
.nav {
  display: flex;
  gap: 16px;
}
```

5.2 Main-axis alignment

`justify-content` controls placement along the main axis.

```
.nav {
  display: flex;
```

```
  justify-content: space-between;
}
```

5.3 Cross-axis alignment

`align-items` controls placement across the flex line.

```
.toolbar {
  min-height: 80px;
  display: flex;
  align-items: center;
}
```

5.4 Direction and wrapping

`flex-direction` changes the axis, and `flex-wrap` lets items move to a new line.

```
.card-list {
  display: flex;
  flex-wrap: wrap;
  gap: 24px;
}

.card {
  flex: 1 1 280px;
}
```

Practice 4: Build a responsive navigation

Create a logo and three navigation links in one flex container. Space them evenly on large screens, center them on smaller screens, and wrap them if there is not enough room.

---

6. Layout with Grid

6.1 Grid columns

CSS Grid is designed for two-dimensional layouts. `grid-template-columns` defines the columns and `gap` creates space between tracks.

```
<section class="gallery">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</section>
```

```css
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```

6.2 Responsive columns with minmax

`minmax()` creates flexible tracks, and `auto-fit` can produce a responsive grid without separate breakpoints.

```css
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}
```

6.3 Named areas

Named grid areas make a page layout easier to read.

```html
<div class="page">
  <header>Header</header>
  <nav>Navigation</nav>
  <main>Main content</main>
  <aside>Sidebar</aside>
  <footer>Footer</footer>
</div>
```

```css
.page {
  display: grid;
  grid-template-areas:
    "header header"
    "nav main"
    "nav footer";
  grid-template-columns: 200px 1fr;
  gap: 16px;
}

.page header { grid-area: header; }
.page nav { grid-area: nav; }
.page main { grid-area: main; }
.page footer { grid-area: footer; }
.page aside { grid-area: sidebar; }
```

---

7. Positioning

7.1 The position property

`position` controls how an element is placed relative to its normal position, a positioned ancestor, or the viewport.

```
.relative {
  position: relative;
  top: 10px;
  left: 20px;
}

.absolute {
  position: absolute;
  top: 0;
  right: 0;
}

.fixed {
  position: fixed;
  bottom: 20px;
  right: 20px;
}

.sticky {
  position: sticky;
  top: 0;
}
```

7.2 Z-index

Z-index controls which positioned element appears in front of another.

```
.overlay {
  position: absolute;
  z-index: 10;
}

.page-content {
  position: relative;
  z-index: 1;
}
```

Practice 5: Build an image badge

Create an image card with a discount badge placed in the top-right corner. Use a positioned ancestor and an absolutely positioned badge, then add a shadow and rounded corners.

8. Responsive design

8.1 The viewport meta tag

Responsive pages need the viewport meta tag in HTML so mobile browsers use the device width.

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

8.2 Media queries

Media queries apply CSS only when a condition is true. Start with mobile styles, then add larger layouts with `min-width` queries.

```
.container {
  width: 100%;
  padding: 16px;
}

@media (min-width: 768px) {
  .container {
    max-width: 720px;
    margin: 0 auto;
  }
}

@media (min-width: 1024px) {
  .container {
    max-width: 960px;
  }
}
```

8.3 Responsive images and typography

Fluid sizes and `clamp()` can scale typography smoothly between minimum and maximum values.

```
img {
  max-width: 100%;
  height: auto;
}

h1 {
  font-size: clamp(2rem, 8vw, 4rem);
}
```

Practice 6: Make a page responsive

Take a page containing a header, two columns, and a card grid. Stack the columns below 768px, keep them beside each other above 768px, and reduce spacing on small screens.

---

9. Transitions, transforms, and animation

9.1 Transitions

A transition animates a property when its value changes, such as during hover.

```
.button {
  background: navy;
  transition: background-color 200ms ease, transform 200ms ease;
}

.button:hover {
  background: royalblue;
  transform: translateY(-2px);
}
```

9.2 Transform

Transform changes the visual position, size, or rotation of an element without changing the normal layout flow.

```
.card:hover {
  transform: scale(1.03);
}

.rotate {
  transform: rotate(45deg);
}
```

9.3 Keyframe animations

Keyframes define what happens during an animation.

```
@keyframes fade-in {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

```
.content {
  animation: fade-in 600ms ease-out both;
}
```

9.4 Reduced motion

Respect users who request reduced motion by disabling or reducing non-essential animation.

```
@media (prefers-reduced-motion: reduce) {
  * {
    scroll-behavior: auto;
    animation-duration: 0.01ms;
    transition-duration: 0.01ms;
  }
}
```

---

10. Forms and accessibility

10.1 Styling form controls

Form controls need consistent spacing, sizing, borders, and clear focus feedback.

```
<label for="email">Email address</label>
<input id="email" name="email" type="email">
<button type="submit">Send</button>
```

```
label {
  display: block;
  margin-bottom: 8px;
}

input,
button {
  width: 100%;
  padding: 12px;
  font: inherit;
}

input {
  border: 1px solid #9ca3af;
  border-radius: 6px;
}

input:focus {
  border-color: #2563eb;
  outline: 3px solid rgba(37, 99, 235, 0.25);
```

```
}

button {
  border: 0;
  color: white;
  background: #2563eb;
  cursor: pointer;
}
```

10.2 States and validation

Use CSS states to show where keyboard focus is and whether a submitted value is invalid.

```
button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

input:invalid {
  border-color: #dc2626;
}

input:focus-visible {
  outline: 3px solid orange;
}
```

Practice 7: Style an accessible form

Style a registration form with labels, full-width fields, clear spacing, a visible focus ring, a disabled button state, and an invalid-field state. Test it using only the keyboard.

---

11. Custom properties and reusable CSS

11.1 CSS variables

Custom properties store reusable values and can change depending on the element or media query.

```
:root {
  --color-primary: #2563eb;
  --color-text: #1f2937;
  --space-medium: 16px;
  --radius: 8px;
}

.button {
  color: white;
  background: var(--color-primary);
  padding: var(--space-medium);
```

```
  border-radius: var(--radius);
}
```

11.2 Component structure

Keep reusable parts consistent by combining a base class with modifier classes.

```
<button class="button button--primary">Save</button>
<button class="button button--secondary">Cancel</button>
```

```
.button {
  padding: 10px 16px;
  border: 1px solid transparent;
  border-radius: 6px;
}

.button--primary {
  color: white;
  background: var(--color-primary);
}

.button--secondary {
  color: var(--color-primary);
  background: white;
  border-color: var(--color-primary);
}
```

---

12. Final project: a one-page business site

Create a complete one-page website using semantic HTML and an external stylesheet:

- Add a header containing your name or business name and a navigation list.
- Add a hero section with a headline, short introduction, and call-to-action button.
- Build an `about` section with text and an image.
- Build a services section containing at least three cards.
- Display the cards with CSS Grid.
- Add a contact form and footer.
- Use Flexbox for navigation and alignment.
- Use custom properties for colors, spacing, and border radius.
- Add hover and focus states.
- Use media queries so the layout works on mobile and desktop.
- Add one short transition or animation and respect `prefers-reduced-motion`.

Your page should look polished at 320px, 768px, and 1200px viewport widths. If all text can be read, controls can be used by keyboard, colors have enough contrast, and layouts do not overflow, you have built a solid

responsive CSS foundation.

Quick reference

Need	CSS
Match elements	`element`, `.class`, `#id`, `[attribute]`
Match nested elements	`.parent .child`, `.parent > .child`
Match interaction states	`:hover`, `:focus-visible`, `:active`, `:disabled`
Box model	`margin`, `border`, `padding`, `width`, `height`, `box-sizing`
Text	`font-family`, `font-size`, `font-weight`, `line-height`, `text-align`
Background	`background-color`, `background-image`, `background-size`
Flex layout	`display: flex`, `justify-content`, `align-items`, `gap`
Grid layout	`display: grid`, `grid-template-columns`, `grid-template-areas`, `gap`
Position	`position`, `top`, `right`, `bottom`, `left`, `z-index`
Responsive rules	`@media`, `min-width`, `max-width`
Motion	`transition`, `transform`, `@keyframes`, `animation`
Reusable values	`:root`, `--custom-property`, `var()`

What next?

Practice by restyling existing HTML pages, inspect layouts in browser developer tools, learn responsive navigation patterns, then move on to JavaScript for interactivity and behavior.